1. Introduction

What is Wakawaka?

Wakawaka is a modern, multi-paradigm programming language developed by **OpenCan.ai**. It combines the safety of Rust, the concurrency of Go, the expressiveness of Python, and the compile-time power of Zig into a single cohesive language. Wakawaka programs can be run interpreted or compiled to native binaries via C99 transpilation.

Safe by Default

Option types prevent null errors. Parameterized queries prevent SQL injection. Shell escaping prevents command injection.

Fast & Compiled

Transpiles to C99 and compiles with gcc/clang for native performance. Also runs interpreted for rapid development.

Expressive

Pattern matching, closures, pipe operator, f-strings, operator overloading, generics, and more.

Concurrent

Goroutines, channels, actors, async/await, and defer for safe resource cleanup.

Integrated

Built-in shell commands, SQL queries, Python interop, web server, and data science libraries.

Batteries Included

Formatter, type checker, REPL, build system, and web server all ship with the toolchain.

Design Philosophy

- **Safety by default** — Memory safety via Option/Result types, no null pointer crashes, injection-safe shell and SQL
- **Zero-cost abstractions** — Generics, closures, and pattern matching compile to efficient native code
- **Gradual typing** — Type inference everywhere, explicit annotations when you want them
- **Multi-paradigm** — Supports OOP (classes, inheritance, interfaces), functional (closures, map/filter/reduce, pipes), and imperative styles
- **Pragmatism** — Python interop for accessing thousands of existing libraries

2. CLI Toolchain

The `waka` command provides the full Wakawaka toolchain.

`waka run <file.wk> [args...]` — Run a Program

Executes a Wakawaka program in interpreted mode.

```
waka run hello.wk
waka run server.wk --port 8080
waka run script.wk arg1 arg2     # args available via sys.argv
```

`waka build <file.wk> [-o output]` — Compile to Native Binary

Transpiles to C99 and compiles with gcc or clang. Produces a standalone executable.

```
waka build hello.wk              # produces ./hello
waka build hello.wk -o myapp     # custom output name
```

> **Requirements:** A C compiler (gcc or clang) must be installed. On Linux/macOS, `-lm` and `-lpthread` are linked automatically.

`waka serve [dir] [--port PORT] [--host HOST]` — Web Server

Starts the built-in web server. Serves static files and executes `.wk` files as dynamic pages.

```
waka serve                       # serve current dir on port 8080
waka serve ./public --port 3000  # custom directory and port
waka serve --host 0.0.0.0       # bind to all interfaces
```

`waka repl` — Interactive REPL

Start an interactive session. Type code, press Enter on a blank line to execute. Type `exit` or Ctrl+D to quit.

```
wk> let x = 42

wk> println(f"x = {x}")
x = 42

wk> [1, 2, 3].map(|n| n * 2)
=> [2, 4, 6]
```

`waka fmt <file.wk> [--check]` — Format Source

Formats Wakawaka source code with consistent style (4-space indentation, K&R braces).

```
waka fmt messy.wk                # format in place
waka fmt messy.wk --check        # check only (exit 1 if needs formatting)
```

`waka check <file.wk>` — Type Check

Parses and type-checks without executing. Useful for CI/pre-commit validation.

```
waka check mylib.wk              # prints "mylib.wk: OK" or error
```

`waka version`

```
waka version                     # Wakawaka 1.0.0, Python 3.x
```

Global Flags

Flag	Description
`--traceback`	Show full Python stack trace on errors (debugging)

3. Quick Start

Hello World

Create a file `hello.wk`:

```
println("Hello, World!");
```

Run it:

```
waka run hello.wk
```

Compile and run:

```
waka build hello.wk -o hello
./hello
```

A Taste of Wakawaka

```
// Variables and types
let name = "Wakawaka";
let version = 0.1;
let features = ["safe", "fast", "expressive"];

// F-string interpolation
println(f"Welcome to {name} v{version}!");

// Functional programming with pipes
let result = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    |> filter(|n| n % 2 == 0)
    |> map(|n| n ** 2)
    |> sum;
println(f"Sum of squares of evens: {result}");  // 220

// Pattern matching
fn classify(n: int) -> str {
    return match n {
        0 => "zero",
        1..=9 => "small",
        10..=99 => "medium",
        _ => "large",
    };
}

// Error handling with Result
fn divide(a: float, b: float) -> Result<float, str> {
    if b == 0.0 { return err("division by zero"); }
    return ok(a / b);
}

match divide(10.0, 3.0) {
    ok(v) => println(f"Result: {v}"),
    err(e) => println(f"Error: {e}"),
}
```

4. Lexical Structure

Comments

```
// Single-line comment

/* Multi-line
   comment */
```

Keywords

The following identifiers are reserved:

```
let    var    const   fn      class    struct   impl     interface
enum   if     else    while   for      do       match    return
break  continue       import  as       in       go       chan
async  await  defer   unsafe  abstract extends  override module
macro  true   false   none    some     ok       err      pub
priv   static new     self    super    and      or       not
is     pyblock        actor   spawn    receive  comptime
```

Numeric Literals

Type	Example	Description
Integer	`42`, `1_000_000`	Decimal integer (underscores for readability)
Hex	`0xFF`, `0x1A2B`	Hexadecimal
Binary	`0b1010`	Binary
Octal	`0o77`	Octal
Float	`3.14`, `1.0e-5`	IEEE 64-bit floating point

String Literals

Type	Syntax	Description
Basic	`"hello\nworld"`	Standard string with escape sequences (`\n \t \r \\ \" \' \0`)
Raw	`r"no\nescape"`	No escape processing
Byte	`b"bytes"`	Byte string
F-string	`f"Hello, {name}!"`	String interpolation with expressions in `{}`
Shell	`$`ls -la {dir}``	Execute shell command, auto-escape variables
SQL	`@sql`SELECT * FROM t WHERE id = {x}``	Parameterized SQL query

F-String Format Specifiers

```
let pi = 3.14159;
println(f"{pi:.2f}");     // "3.14" (2 decimal places)
println(f"{255:x}");      // "ff" (hex)
println(f"{10:b}");       // "1010" (binary)
println(f"{42:o}");       // "52" (octal)
```

5. Variables & Constants

Declarations

```
let x = 42;                     // immutable, type inferred
var y = "hello";                // mutable
const PI = 3.14159;             // compile-time constant

let z: int = 0;                 // explicit type annotation
var items: []str = [];          // typed mutable array
let maybe: ?str = none;         // optional type
```

> **Immutability:** Variables declared with `let` cannot be reassigned. Use `var` when you need to modify the value

> after declaration.

Type Inference

The compiler infers types from the right-hand side expression. Explicit annotations are optional but allowed everywhere.

```
let a = 42;          // inferred: int
let b = 3.14;        // inferred: float
let c = true;        // inferred: bool
let d = "hello";     // inferred: str
let e = [1, 2, 3];   // inferred: []int
let f = {"k": 1};    // inferred: map[str]int
```

6. Types

Primitive Types

Type	Description	Example
`int`	64-bit signed integer	`42`
`float`	64-bit IEEE floating point	`3.14`
`bool`	Boolean	`true`, `false`
`str`	UTF-8 string	`"hello"`
`byte`	Single byte (u8)	
`rune`	Unicode code point	

Sized Integer Types

Signed: `i8`, `i16`, `i32`, `i64`. Unsigned: `u8`, `u16`, `u32`, `u64`. Sized floats: `f32`, `f64`.

Collection Types

Type	Syntax	Example
Array (dynamic)	`[]T`	`[1, 2, 3]`
Array (fixed)	`[N]T`	`[10]int`
Map	`map[K]V`	`{"a": 1, "b": 2}`
Tuple	`(T1, T2, ...)`	`(1, "hello", 3.14)`

Option Type

Represents a value that may or may not exist. Prevents null pointer errors.

```
let maybe: ?int = some(42);
let nothing: ?str = none;

match maybe {
    some(v) => println(f"Got: {v}"),
    none    => println("Nothing"),
}
```

Result Type

Represents either success or failure. Forces explicit error handling.

```
let good: Result<int, str> = ok(42);
let bad: Result<int, str> = err("failed");

match good {
    ok(v)  => println(f"Success: {v}"),
    err(e) => println(f"Error: {e}"),
}
```

Channel Type

```
let ch = chan<int>(10);    // buffered channel with capacity 10
```

Function Types

```
let f: fn(int, int) -> int = |a, b| a + b;
```

7. Operators

Arithmetic Operators

Operator	Description	Example
`+`	Addition / string concatenation	`3 + 4` → `7`
`-`	Subtraction	`10 - 3` → `7`
`*`	Multiplication	`6 * 7` → `42`
`/`	Division	`10 / 3` → `3`
`%`	Modulo	`10 % 3` → `1`
`**`	Exponentiation	`2 ** 10` → `1024`

Comparison Operators

Operator	Description
`==`, `!=`	Equal, not equal
`<`, `>`	Less than, greater than
`<=`, `>=`	Less or equal, greater or equal

Logical Operators

Operator	Keyword	Description
`&&`	`and`	Logical AND
`||`	`or`	Logical OR
`!`	`not`	Logical NOT

Bitwise Operators

Operator	Description
`&`	Bitwise AND
`|`	Bitwise OR
`^`	Bitwise XOR
`~`	Bitwise NOT
`<<`	Left shift
`>>`	Right shift

Assignment Operators

`=`, `+=`, `-=`, `*=`, `/=`, `%=`, `**=`, `&=`, `|=`, `^=`, `<<=`, `>>=`

Special Operators

Operator	Name	Description	Example
`|>`	Pipe	Passes left value as first argument to right function	`data |> filter(f) |> map(g)`
`?.`	Optional chain	Safe member access, returns `none` if object is `none`	`obj?.field`
`??`	Null coalesce	Returns left if not `none`, else right	`val ?? default`
`?`	Propagate	Unwraps Result, returns early if `err`	`risky()?`
`..`	Range (exclusive)	Creates range [start, end)	`0..10`
`..=`	Range (inclusive)	Creates range [start, end]	`0..=10`
`as`	Cast	Type conversion	`x as float`
`<-`	Channel send	Send value to channel	`ch <- 42`
`? :`	Ternary	Conditional expression	`x > 0 ? "pos" : "neg"`

8. Control Flow

If / Else

```
if x > 0 {
    println("positive");
} else if x < 0 {
    println("negative");
} else {
    println("zero");
}
```

While Loop

```
var count = 0;
while count < 5 {
    println(f"count = {count}");
    count += 1;
}
```

Do-While Loop

```
var attempt = 0;
do {
    attempt += 1;
    println(f"Attempt {attempt}");
} while attempt < 3;
```

For-In Loop

```
// Range iteration
for i in 0..5 {
    print(f"{i} ");        // 0 1 2 3 4
}

// Inclusive range
for i in 1..=5 {
    print(f"{i} ");        // 1 2 3 4 5
}

// Collection iteration
let names = ["Alice", "Bob", "Carol"];
for name in names {
    println(f"Hello, {name}!");
}
```

C-Style For Loop

```
for (let i = 0; i < 10; i += 1) {
    print(f"{i} ");
}
```

Labeled Break & Continue

```
outer:
for i in 0..10 {
    for j in 0..10 {
        if i + j > 15 {
            break outer;       // breaks out of both loops
        }
    }
}
```

Match (Pattern Matching)

```
match value {
    0        => println("zero"),
    1..=9    => println("small"),
    10..=99  => println("medium"),
    _        => println("large"),
}
```

See Pattern Matching for full details.

Defer

Deferred statements execute in reverse order when the enclosing function returns.

```
fn process() {
    let f = open("data.txt");
    defer close(f);           // always runs on function exit
    defer println("cleanup");

    // ... work with file ...
}
// On exit: prints "cleanup", then closes file
```

9. Functions

Basic Declaration

```
fn add(a: int, b: int) -> int {
    return a + b;
}

fn greet(name: str) {
    println(f"Hello, {name}!");
}
```

Multiple Return Values

```
fn divmod(a: int, b: int) -> (int, int) {
    return a / b, a % b;
}

let q, r = divmod(17, 5);   // q=3, r=2
```

Default Parameters

```
fn greet(name: str, greeting: str = "Hello") -> str {
    return f"{greeting}, {name}!";
}

greet("Alice");                  // "Hello, Alice!"
greet("Bob", greeting: "Hi");    // "Hi, Bob!"
```

Variadic Functions

```
fn sum(..nums: []int) -> int {
    return nums.reduce(|a, b| a + b, 0);
}

sum(1, 2, 3, 4, 5);   // 15
```

Closures (Anonymous Functions)

```
let double = |x| x * 2;
let add = |a, b| a + b;

// Multi-line closure
let process = |data| {
    let filtered = data.filter(|x| x > 0);
    return filtered.sum();
};

// Used in higher-order functions
let evens = [1, 2, 3, 4, 5].filter(|n| n % 2 == 0);
let doubled = evens.map(|n| n * 2);
```

Higher-Order Functions

```
fn make_adder(n: int) -> fn(int) -> int {
    return |x| x + n;
}

fn compose(f: fn, g: fn) -> fn {
    return |x| f(g(x));
}

let add5 = make_adder(5);
let mul3 = |x| x * 3;
let add5_then_mul3 = compose(mul3, add5);
println(add5_then_mul3(10));   // 45
```

Generic Functions

```
fn max<T: Ordered>(a: T, b: T) -> T {
    return a > b ? a : b;
}

fn swap<T>(a: T, b: T) -> (T, T) {
    return b, a;
}
```

Async Functions

```
async fn fetch_data(url: str) -> Result<str, str> {
    let resp = await http.get(url);
    return ok(resp);
}

let data = await fetch_data("https://api.example.com");
```

10. Classes & OOP

Structs (Data Only)

```
struct Point {
    x: float,
    y: float,
}

let p = Point { x: 1.0, y: 2.0 };
println(f"({p.x}, {p.y})");
```

Impl Blocks (Methods for Structs)

```
impl Point {
    fn distance(self, other: Point) -> float {
        let dx = self.x - other.x;
        let dy = self.y - other.y;
        return math.sqrt(dx*dx + dy*dy);
    }
}
```

Classes

```
class Animal {
    var name: str;
    var sound: str;

    fn new(name: str, sound: str) -> Animal {
        return Animal { name: name, sound: sound };
    }

    fn speak(self) -> str {
        return f"{self.name} says {self.sound}!";
    }
}
```

Inheritance

```
class Dog extends Animal {
    var breed: str;

    fn new(name: str, breed: str) -> Dog {
        return Dog { name: name, sound: "woof", breed: breed };
    }

    override fn speak(self) -> str {
        return f"{self.name} the {self.breed} barks!";
    }
}
```

Abstract Classes

```
abstract class Shape {
    var color: str;

    abstract fn area(self) -> float;
    abstract fn perimeter(self) -> float;

    fn describe(self) -> str {
        return f"[{self.color}] area={self.area()}";
    }
}

class Circle extends Shape {
    var radius: float;

    fn new(r: float, color: str) -> Circle {
        return Circle { radius: r, color: color };
    }

    override fn area(self) -> float {
        return math.PI * self.radius ** 2;
    }

    override fn perimeter(self) -> float {
        return 2.0 * math.PI * self.radius;
    }
}
```

Interfaces

```
interface Drawable {
    fn draw(self);
    fn area(self) -> float;
}

interface Comparable {
    fn compareTo(self, other: any) -> int;
}
```

Operator Overloading

```
class Vec2 {
    var x: float;
    var y: float;

    fn operator+(self, other: Vec2) -> Vec2 {
        return Vec2 { x: self.x + other.x, y: self.y + other.y };
    }

    fn operator==(self, other: Vec2) -> bool {
        return self.x == other.x and self.y == other.y;
    }

    fn operator<(self, other: Vec2) -> bool {
        return self.x < other.x;
    }
}

let a = Vec2 { x: 1.0, y: 2.0 };
let b = Vec2 { x: 3.0, y: 4.0 };
let c = a + b;    // Vec2 { x: 4.0, y: 6.0 }
```

Generic Classes

```
class Stack<T> {
    var items: []T;

    fn new() -> Stack<T> {
        return Stack { items: [] };
    }

    fn push(var self, item: T) {
        self.items.push(item);
    }

    fn pop(var self) -> ?T {
        if self.items.isEmpty() { return none; }
        return some(self.items.pop());
    }

    fn peek(self) -> ?T {
        if self.items.isEmpty() { return none; }
        return some(self.items.last());
    }

    fn size(self) -> int { return self.items.len(); }
}

let stack = Stack<int>.new();
stack.push(1);
stack.push(2);
let val = stack.pop();   // some(2)
```

11. Enums

Simple Enums

```
enum Color { Red, Green, Blue }
enum Status { Pending, Active, Completed }

let c = Color.Red;
```

Enums with Associated Data

```
enum Shape {
    Circle(float),
    Rectangle(float, float),
    Triangle(float, float, float),
}

let s = Shape.Circle(5.0);
let r = Shape.Rectangle(4.0, 6.0);
```

Pattern Matching on Enums

```
match shape {
    Shape.Circle(r) => println(f"Circle with radius {r}"),
    Shape.Rectangle(w, h) => println(f"Rectangle {w}x{h}"),
    Shape.Triangle(a, b, c) => println(f"Triangle {a},{b},{c}"),
}
```

12. Pattern Matching

The `match` expression supports many pattern types:

Literal Patterns

```
match n {
    0 => "zero",
    1 => "one",
    42 => "the answer",
    _ => "other",
}
```

Range Patterns

```
match score {
    0..=59 => "F",
    60..=69 => "D",
    70..=79 => "C",
    80..=89 => "B",
    90..=100 => "A",
}
```

Binding Patterns

```
match maybe_value {
    some(x) => println(f"Got {x}"),
    none => println("Nothing"),
}

match result {
    ok(data) => process(data),
    err(msg) => println(f"Error: {msg}"),
}
```

Tuple Patterns

```
match pair {
    (0, y) => println(f"x is zero, y is {y}"),
    (x, 0) => println(f"x is {x}, y is zero"),
    (x, y) => println(f"({x}, {y})"),
}
```

Struct Patterns

```
match point {
    Point { x: 0.0, y: 0.0 } => println("origin"),
    Point { x, y } => println(f"({x}, {y})"),
}
```

Or Patterns

```
match day {
    "Saturday" | "Sunday" => println("weekend"),
    _ => println("weekday"),
}
```

Guard Expressions

```
match n {
    x if x > 0 => println("positive"),
    x if x < 0 => println("negative"),
    _ => println("zero"),
}
```

Wildcard

```
match value {
    _ => println("matches anything"),
}
```

13. Error Handling

Result Type

```
fn parse_int(s: str) -> Result<int, str> {
    let n = int(s);
    if n == 0 and s != "0" {
        return err(f"cannot parse '{s}' as int");
    }
    return ok(n);
}

match parse_int("42") {
    ok(n) => println(f"Parsed: {n}"),
    err(e) => println(f"Error: {e}"),
}
```

Error Propagation (?)

```
fn process(input: str) -> Result<int, str> {
    let a = parse_int(input)?;     // returns err early if parse fails
    let b = divide(a, 2)?;        // returns err early if divide fails
    return ok(b);
}
```

Option Type

```
fn find(items: []str, target: str) -> ?int {
    for i in 0..items.len() {
        if items[i] == target { return some(i); }
    }
    return none;
}

let idx = find(names, "Alice");
match idx {
    some(i) => println(f"Found at index {i}"),
    none => println("Not found"),
}
```

Optional Chaining

```
let city = user?.address?.city;   // none if any step is none
```

Null Coalescing

```
let name = user?.name ?? "Anonymous";
```

Type Checking Builtins

```
isNone(value)    // true if value is none
isSome(value)    // true if value is some(...)
isOk(value)      // true if value is ok(...)
isErr(value)     // true if value is err(...)
```

14. Concurrency

Goroutines

Spawn lightweight background threads with `go`:

```
fn worker(id: int) {
    println(f"Worker {id} running");
}

go worker(1);
go worker(2);
go worker(3);
sleep(1.0);    // wait for goroutines to finish
```

Channels

```
let ch = chan<str>(10);      // buffered channel, capacity 10

// Send
go fn() {
    ch <- "hello";
    ch <- "world";
}();

// Receive
let msg1 = recv(ch);
let msg2 = recv(ch);
println(f"{msg1} {msg2}");  // "hello world"
```

Producer-Consumer Pattern

```
fn producer(ch: chan, id: int) {
    for i in 0..3 {
        ch <- f"worker-{id} item-{i}";
    }
}

let ch = chan<str>(20);
go producer(ch, 1);
go producer(ch, 2);
go producer(ch, 3);

sleep(0.5);
// Receive all messages...
```

Actor Model

```
actor Counter {
    var count: int;

    fn new(initial: int) -> Counter {
        return Counter { count: initial };
    }

    fn increment(var self, n: int) {
        self.count += n;
    }

    fn get(self) -> int {
        return self.count;
    }
}

let c = spawn Counter.new(0);
// Actors provide thread-safe state management
```

Defer (Resource Cleanup)

```
fn with_resource() {
    let res = acquire_resource();
    defer release(res);            // always runs on exit

    // If an error occurs here, resource is still released
    process(res);
}

// Multiple defers execute in reverse (LIFO) order
fn multi_defer() {
    defer println("third");
    defer println("second");
    defer println("first");
}
// Prints: first, second, third
```

15. Modules & Imports

Native Wakawaka Imports

```
// mymath.wk
fn add(a: int, b: int) -> int {
    return a + b;
}
let PI = 3.14159;
```

```
// main.wk
import "mymath.wk" as mm;

fn main() {
    println(f"add(3, 4) = {mm.add(3, 4)}");
    println(f"PI = {mm.PI}");
}
main();
```

> **Module imports work in both interpreted and compiled mode.** The compiler creates a namespace map for each imported module, preventing name collisions.

Python Imports

```
import python "numpy" as np;
import python "json" as pyjson;
import python "datetime" as dt;

let arr = np.array([1, 2, 3, 4, 5]);
let mean = np.mean(arr);
println(f"Mean: {mean}");
```

Module Declarations

```
module Utils {
    fn double(x: int) -> int {
        return x * 2;
    }
}

Utils.double(21);    // 42
```

16. Shell Integration

Shell Command Literals

```
let cwd = $`pwd`.trim();
let files = $`ls -la`.lines();
let date = $`date`.trim();
println(f"Current directory: {cwd}");
```

Safe Variable Interpolation

Variables in shell literals are automatically escaped, preventing command injection:

```
let filename = "my file.txt";
let content = $`cat {filename}`;       // safe: spaces handled

let user_input = "'; rm -rf /; echo '";
$`echo {user_input}`;                  // safe: no injection
```

Processing Shell Output

```
// Get lines as a list
let log_lines = $`cat /var/log/app.log`.trim().lines();

// Count files
let wk_files = $`find . -name "*.wk"`.trim().lines();
println(f"Found {wk_files.len()} .wk files");

// Chain with string methods
let first_five = $`ls -la`.lines().slice(0, 5);
for line in first_five {
    println(f"  {line}");
}
```

> **Compiled mode note:** Shell commands are executed via `popen()` in the C runtime, capturing stdout. Stderr is not captured by default.

17. SQL Integration

Connecting to a Database

```
let db = sql.connect("sqlite:myapp.db");
```

Parameterized Queries

All `{variables}` in `@sql` literals are parameterized — SQL injection is impossible:

```
let min_age = 30;
let users = db.query(@sql`
    SELECT * FROM users
    WHERE age >= {min_age}
    ORDER BY name
`);
```

Insert / Update / Delete

```
let name = "Alice";
let email = "alice@example.com";
db.exec(@sql`
    INSERT INTO users (name, email)
    VALUES ({name}, {email})
`);

let new_score = 95.5;
let id = 1;
db.exec(@sql`UPDATE users SET score = {new_score} WHERE id = {id}`);
```

Transactions

```
db.transaction(|tx| {
    tx.exec(@sql`UPDATE accounts SET balance -= 100 WHERE id = 1`);
    tx.exec(@sql`UPDATE accounts SET balance += 100 WHERE id = 2`);
    // Auto-commit on success, auto-rollback on error
});
```

Accessing Results

```
let users = db.query(@sql`SELECT id, name, email FROM users`);
for user in users {
    println(f"[{user.id}] {user.name} ({user.email})");
}
```

18. Tensor Computing

Wakawaka has first-class tensor support for numerical and machine-learning workloads. Tensors are n-dimensional arrays of double-precision floats with broadcasting, element-wise arithmetic, and linear algebra operations built in. No external dependencies required.

Creating Tensors

```
// From explicit data
let a = tensor.from([1.0, 2.0, 3.0, 4.0]);
let matrix = tensor.from([[1, 2], [3, 4]]);

// Factory functions
let z = tensor.zeros([3, 3]);          // 3x3 zeros
let o = tensor.ones([2, 4]);           // 2x4 ones
let I = tensor.eye(4);                 // 4x4 identity
let r = tensor.rand([3, 3]);           // 3x3 random [0,1)
let seq = tensor.arange(0, 10, 2);     // [0, 2, 4, 6, 8]
let lin = tensor.linspace(0, 1, 5);    // [0, 0.25, 0.5, 0.75, 1.0]
```

Arithmetic & Broadcasting

```
let a = tensor.from([1, 2, 3]);
let b = tensor.from([10, 20, 30]);

let sum = a + b;        // [11, 22, 33]
let diff = b - a;       // [9, 18, 27]
let prod = a * b;       // [10, 40, 90]  element-wise
let quot = b / a;       // [10, 10, 10]
```

Reductions & Properties

```
let t = tensor.from([3.0, 1.0, 4.0, 1.0, 5.0]);

println(t.sum());       // 14.0
println(t.mean());      // 2.8
println(t.min());       // 1.0
println(t.max());       // 5.0
println(t.argmax());    // 4  (index of 5.0)
println(t.shape());     // [5]
println(t.ndim);        // 1
println(t.size);        // 5
```

Reshaping & Linear Algebra

```
let a = tensor.arange(0, 6);
let m = a.reshape([2, 3]);     // 2x3 matrix
let mt = m.transpose();        // 3x2 matrix
let flat = m.flatten();        // back to 1D

// Matrix multiplication
let A = tensor.from([[1, 2], [3, 4]]);
let B = tensor.from([[5, 6], [7, 8]]);
let C = A.matmul(B);           // [[19, 22], [43, 50]]

// Dot product
let v1 = tensor.from([1, 2, 3]);
let v2 = tensor.from([4, 5, 6]);
println(v1.dot(v2));            // 32.0
```

19. Automatic Differentiation

The `ad` module provides forward-mode automatic differentiation using dual numbers. Compute exact gradients of arbitrary functions — no numerical approximation, no symbolic math engine.

Dual Numbers

```
// Create a dual number: value + derivative
let x = ad.dual(3.0, 1.0);    // x = 3, dx/dx = 1

// Arithmetic propagates derivatives automatically
let y = x * x;                // value = 9, deriv = 6  (d/dx of x^2 at x=3)
println(ad.value(y));         // 9.0
println(ad.deriv(y));         // 6.0
```

Computing Gradients

```
// Define a function
fn f(x) { return x * x * x - 2 * x + 5; }

// Compute derivative at x = 3
let grad = ad.grad(f, 3.0);
println(grad);                // 25.0  (3*9 - 2 = 25)

// Works with transcendental functions
fn g(x) { return ad.sin(x) * ad.exp(x); }
let dg = ad.grad(g, 0.0);
println(dg);                  // 1.0  (sin(0)*exp(0) + cos(0)*exp(0))
```

Dual-Aware Math Functions

```
let x = ad.dual(1.0, 1.0);

ad.sin(x);     // sin with chain rule
ad.cos(x);     // cos with chain rule
ad.exp(x);     // exp with chain rule
ad.log(x);     // natural log with chain rule
ad.sqrt(x);    // sqrt with chain rule
ad.pow(x, 3);  // power with chain rule
```

> **Use case:** Training neural networks, physics simulations, optimization — anywhere you need exact gradients without hand-deriving formulas.

20. ML Pipelines

The `pipeline` module provides lazy, chainable data pipelines for preprocessing, batching, and transforming datasets. Transformations are deferred until a terminal operation executes.

Creating & Transforming

```
let data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

let result = pipeline.from(data)
    .filter(|x| x % 2 == 0)       // [2, 4, 6, 8, 10]
    .map(|x| x * x)               // [4, 16, 36, 64, 100]
    .collect();

println(result);                  // [4, 16, 36, 64, 100]
```

Batching & Shuffling

```
let samples = pipeline.from(training_data)
    .shuffle()                      // randomize order
    .batch(32)                      // group into batches of 32
    .take(10)                       // first 10 batches
    .collect();
```

Terminal Operations

```
let data = pipeline.from([1, 2, 3, 4, 5]);

let n = data.count();                           // 5
let total = data.reduce(|acc, x| acc + x, 0);  // 15
data.forEach(|x| println(x));                   // prints each
```

Chaining with Tensors

```
// Load data, normalize, batch, and train
let batches = pipeline.from(raw_samples)
    .map(|s| tensor.from(s))
    .map(|t| t / 255.0)           // normalize
    .shuffle()
    .batch(64)
    .collect();
```

21. Model I/O

The `model` module handles saving and loading trained models in Wakawaka's native WKTM (Wakawaka Tensor Model) format, plus importing weights from GGUF (llama.cpp) and ONNX formats.

Saving & Loading

```
// Save model weights
let weights = {
    "layer1": tensor.rand([784, 128]),
    "bias1": tensor.zeros([128]),
    "layer2": tensor.rand([128, 10]),
    "bias2": tensor.zeros([10]),
};

model.save(weights, "my_model.wktm");

// Load back
let loaded = model.load("my_model.wktm");
let l1 = loaded["layer1"];   // tensor restored
```

Model Metadata

```
model.setMeta(weights, "name", "MNIST Classifier");
model.setMeta(weights, "accuracy", "97.3%");
model.setMeta(weights, "epochs", "50");
model.save(weights, "my_model.wktm");

// Read metadata
let m = model.load("my_model.wktm");
println(model.meta(m, "name"));       // "MNIST Classifier"
println(model.meta(m));               // all metadata as map
```

Importing External Formats

```
// Load GGUF weights (llama.cpp models)
let llama = model.loadGGUF("llama-7b.gguf");
for name in llama.keys() {
    let t = llama[name];
    println(f"{name}: shape={t.shape()}, size={t.size}");
}

// Load ONNX weights
let onnx = model.loadONNX("resnet50.onnx");
println(f"Loaded {onnx.keys().len()} tensors");
```

22. GPU Computing

The `gpu` module provides access to GPU hardware via OpenCL (Khronos open standard, royalty-free). Run compute kernels on any OpenCL-capable device — NVIDIA, AMD, Intel, or Apple GPUs.

Checking Availability

```
if gpu.available() {
    let devices = gpu.devices();
    for d in devices {
        println(f"GPU: {d}");
    }
} else {
    println("No GPU / OpenCL not installed");
}
```

Running a Kernel

```
let kernel = "
__kernel void vector_add(
    __global const float* A,
    __global const float* B,
    __global float* C,
    const int N)
{
    int i = get_global_id(0);
    if (i < N) C[i] = A[i] + B[i];
}
";

let result = gpu.run(kernel);
```

> **Platform note:** Requires OpenCL runtime installed on the system. Falls back gracefully if unavailable —

> `gpu.available()` returns `false`.

23. Foreign Modules (FFI)

Wakawaka can call into Python, Java, and Node.js at runtime via dynamic linking (`dlopen`). No build-time dependency — if the runtime is installed, it works. If not, the module gracefully reports unavailable. This approach carries **zero licensing obligations**.

Python

```
if py.available() {
    let np = py.import("numpy");
    let result = py.call(np, "array", [1, 2, 3]);
    println(result);

    // Evaluate arbitrary Python
    let pi = py.eval("__import__('math').pi");
    println(f"Pi = {pi}");
}
```

Java / JVM

```
if jvm.available() {
    let System = jvm.import("java.lang.System");
    let time = jvm.call(System, "currentTimeMillis");
    println(f"Java time: {time}");
}
```

> **Requires:** `JAVA_HOME` set and `libjvm.so` accessible.

Node.js

```
if node.available() {
    let _ = node.require("lodash");
    let result = node.eval("require('os').cpus().length");
    println(f"CPU cores: {result}");
}
```

C / C++ Interop

Compiled Wakawaka transpiles to C99 and links against the Wakawaka runtime. You can extend it directly:

```
// 1. Write a C function using the WkVal ABI:
//    WkVal my_func(WkVal *args, int argc, WkFunc *fn);
//
// 2. Register it in wk_runtime_init() or your own init function
//
// 3. Compile alongside your .wk program:
//    gcc my_ext.c program.c runtime/waka_runtime.c -o program -lm
```

Since Wakawaka compiles to plain C, it interoperates with any C/C++ library. Link against SQLite, OpenSSL, BLAS, CUDA — anything with a C ABI.

Interpreter Mode (pyblock)

In interpreted mode, you can also use inline Python blocks for quick prototyping:

```
let numbers = [1, 2, 3, 4, 5];
pyblock {
    import statistics
    mean_val = statistics.mean(numbers)
    result = f"Mean = {mean_val}"
}
println(result);    // "Mean = 3"
```

> **Note:** `pyblock` is only available in interpreted mode (`waka run`). Use `py.call()` for compiled mode.

Platform Support

Platform	Interpreter	Compiled	FFI (dlopen)
Linux	Full	Full	Full
macOS	Full	Full	Full
Windows	Full	MinGW/MSYS2	Planned
AIX / Solaris / HP-UX	Full	C99 + POSIX	dlopen compatible

24. Built-in Web Server

Starting the Server

```
waka serve ./public --port 3000
```

Template Syntax

Files with `.wk` extension are executed as dynamic pages when served:

```
<!DOCTYPE html>
<html>
<body>
  <h1>Hello, <?= request.GET.get("name") ?? "World" ?>!</h1>

  <?wk
  let items = ["Apple", "Banana", "Cherry"];
  ?>

  <ul>
  <?wk for item in items { ?>
    <li><?= item ?></li>
  <?wk } ?>
  </ul>
</body>
</html>
```

Template Tags

Tag	Description
`<?= expr ?>`	Evaluate expression and output result
`<?wk code ?>`	Execute Wakawaka code block

Request Object

Property	Description
`request.method`	HTTP method (GET, POST)
`request.path`	URL path
`request.GET`	Query parameters (map)
`request.POST`	Form/JSON body data (map)
`request.headers`	Request headers (map)
`request.body`	Raw request body (string)
`request.cookies`	Parsed cookies (map)

PHP-Compatible Globals

For familiarity, these aliases are available: `$_GET`, `$_POST`, `$_COOKIE`, `$_SERVER`.

Response Functions

Function	Description
`echo(str)`	Write to response body
`header(name, value)`	Set response header
`status(code)`	Set HTTP status code
`die([msg])`	Stop execution, optionally write message

Routing

URLs map directly to filesystem paths. The server looks for `index.wk`, `index.html`, or `index.htm` in directories. Static files (.html, .css, .js, .png, .jpg, etc.) are served directly. Sensitive files (.py, .env, .key, .pem) are denied.

25. Builtin Functions Reference

I/O Functions

Function	Description
`println(...args)`	Print arguments with trailing newline
`print(...args)`	Print arguments without newline
`eprintln(...args)`	Print to stderr with newline
`readln([prompt])`	Read a line from stdin (optional prompt)

Type Conversion

Function	Description
`str(value)`	Convert to string
`int(value)`	Convert to integer
`float(value)`	Convert to float
`bool(value)`	Convert to boolean

Type Checking

Function	Description
`typeof(value)`	Returns type name as string ("int", "str", "list", etc.)
`len(value)`	Length of string, list, map, tuple, or range
`isNone(value)`	True if value is `none`
`isSome(value)`	True if value is `some(...)`
`isOk(value)`	True if value is `ok(...)`
`isErr(value)`	True if value is `err(...)`

Aggregation

Function	Description
`sum(list)`	Sum all numeric elements
`min(...args)` or `min(list)`	Minimum value
`max(...args)` or `max(list)`	Maximum value

Functional

Function	Description
`map(list, fn)`	Apply function to each element, return new list
`filter(list, fn)`	Return elements where function returns truthy
`reduce(fn, list [, init])`	Reduce list using accumulator function
`sorted(list [, key])`	Return new sorted list
`reversed(list)`	Return new reversed list
`any(list [, fn])`	True if any element is truthy (or passes predicate)
`all(list [, fn])`	True if all elements are truthy (or pass predicate)
`zip(list1, list2)`	Pair elements into tuples
`enumerate(list)`	Return (index, element) tuples
`range(end)` or `range(start, end)`	Create integer range [start, end)

Utility

Function	Description
`copy(value)`	Deep copy of any value (recursive for nested structures)
`chr(n)`	Integer to character string
`ord(s)`	First character to integer (byte value)
`hash(value)`	Hash code of value
`repr(value)`	Debug representation string
`sleep(seconds)`	Sleep for given duration (supports fractions)
`exit([code])`	Exit program with status code (default 0)
`assert(cond [, msg])`	Panic if condition is falsy
`panic(msg)`	Immediately halt with error message

26. String Methods

Method	Description	Returns
`.len()`	String length	`int`
`.upper()`	Uppercase copy	`str`
`.lower()`	Lowercase copy	`str`
`.trim()`	Remove leading/trailing whitespace	`str`
`.isEmpty()`	True if empty string	`bool`
`.toInt()`	Parse as integer	`int`
`.toFloat()`	Parse as float	`float`
`.split([sep])`	Split by separator (default: space)	`[]str`
`.contains(sub)`	True if contains substring	`bool`
`.startsWith(prefix)`	True if starts with prefix	`bool`
`.endsWith(suffix)`	True if ends with suffix	`bool`
`.replace(old, new)`	Replace all occurrences	`str`
`.indexOf(sub)`	Index of first occurrence (-1 if not found)	`int`
`.repeat(n)`	Repeat string n times	`str`
`.lines()`	Split by newlines	`[]str`
`.chars()`	List of single characters	`[]str`
`.slice(start [, end])`	Substring (negative indices supported)	`str`
`.bytes()`	List of byte values	`[]int`

Examples

```
let s = "  Hello, World!  ";
println(s.trim());            // "Hello, World!"
println(s.trim().upper());    // "HELLO, WORLD!"
println(s.trim().split(", "));// ["Hello", "World!"]
println("abc".repeat(3));     // "abcabcabc"
println("hello".contains("ell")); // true
println("hello"[1:4]);        // "ell" (slice syntax)
```

27. List Methods

Method	Description	Mutates?

`.len()`	Number of elements	No
`.isEmpty()`	True if empty	No
`.push(item)`	Append element	Yes
`.append(item)`	Alias for push	Yes
`.pop()`	Remove and return last element	Yes
`.insert(idx, item)`	Insert at index	Yes
`.remove(item)`	Remove first occurrence	Yes
`.contains(item)`	True if list contains item	No
`.indexOf(item)`	Index of first occurrence (-1 if not found)	No
`.first()`	First element or none	No
`.last()`	Last element or none	No
`.clear()`	Remove all elements	Yes
`.reverse()`	Reverse in place	Yes
`.sort([key])`	Sort in place (optional key function)	Yes
`.sorted([key])`	Return new sorted list	No
`.join(sep)`	Join elements as string	No
`.map(fn)`	Apply function, return new list	No
`.filter(fn)`	Keep matching elements, return new list	No
`.reduce(fn [, init])`	Reduce to single value	No
`.forEach(fn)`	Call function on each element	No
`.any([fn])`	True if any element is truthy / passes predicate	No
`.all([fn])`	True if all elements truthy / pass predicate	No
`.sum()`	Sum of numeric elements	No
`.min()`	Minimum element	No
`.max()`	Maximum element	No
`.slice(start [, end])`	Return sublist	No
`.unique()`	Remove duplicates (preserves order)	No

`.flat()`	Flatten one level of nesting	No
`.zip(other)`	Pair with another list	No
`.enumerate()`	Return (index, element) tuples	No
`.count([val])`	Count occurrences (or length if no arg)	No
`.extend(other)`	Append all elements from another list	Yes

Slice Syntax

```
let nums = [10, 20, 30, 40, 50, 60, 70, 80];
nums[1:4]      // [20, 30, 40]
nums[:3]       // [10, 20, 30]
nums[5:]       // [60, 70, 80]
nums[::2]      // [10, 30, 50, 70]     (every 2nd element)
nums[1:6:2]    // [20, 40, 60]         (with step)
```

Examples

```
let items = [3, 1, 4, 1, 5, 9, 2, 6];

items.sorted()                    // [1, 1, 2, 3, 4, 5, 6, 9]
items.filter(|x| x > 3)          // [4, 5, 9, 6]
items.map(|x| x * 2)             // [6, 2, 8, 2, 10, 18, 4, 12]
items.reduce(|a, b| a + b, 0)    // 31
items.unique()                    // [3, 1, 4, 5, 9, 2, 6]
[1, 2, 3].join(", ")             // "1, 2, 3"
```

28. Map Methods

Method	Description	Mutates?
`.len()`	Number of key-value pairs	No
`.isEmpty()`	True if empty	No
`.has(key)`	True if key exists	No
`.get(key [, default])`	Get value by key (or default/none)	No
`.set(key, value)`	Set key-value pair	Yes
`.delete(key)`	Remove key	Yes
`.keys()`	List of all keys	No
`.values()`	List of all values	No
`.entries()`	List of (key, value) tuples	No
`.clear()`	Remove all entries	Yes
`.merge(other)`	Merge another map into this one	Yes

Examples

```
let scores = {"Alice": 95, "Bob": 87, "Carol": 92};

println(scores.keys());           // ["Alice", "Bob", "Carol"]
println(scores.get("Alice"));     // 95
println(scores.get("Dave", 0));   // 0 (default)
println(scores.has("Bob"));       // true

scores.set("Dave", 88);
scores.delete("Bob");

for entry in scores.entries() {
    println(f"{entry[0]}: {entry[1]}");
}
```

29. Math Module

Constants

Name	Value
`math.PI`	3.14159265358979...
`math.E`	2.71828182845905...
`math.TAU`	6.28318530717959... (2 * PI)
`math.INF`	Positive infinity
`math.NAN`	Not a number

Functions

Function	Description
`math.sqrt(x)`	Square root
`math.abs(x)`	Absolute value
`math.floor(x)`	Floor (largest integer <= x)
`math.ceil(x)`	Ceiling (smallest integer >= x)
`math.pow(base, exp)`	Power (base^exp)
`math.sin(x)`	Sine (radians)
`math.cos(x)`	Cosine (radians)
`math.tan(x)`	Tangent (radians)
`math.asin(x)`	Arcsine
`math.acos(x)`	Arccosine
`math.atan(x)`	Arctangent
`math.atan2(y, x)`	Two-argument arctangent
`math.exp(x)`	e^x
`math.log(x)`	Natural logarithm
`math.log2(x)`	Base-2 logarithm
`math.log10(x)`	Base-10 logarithm

Examples

```
println(math.sqrt(16));        // 4.0
println(math.PI);              // 3.14159265358979
println(math.sin(math.PI/2));  // 1.0
println(math.log(math.E));     // 1.0
println(math.floor(3.7));      // 3.0
println(math.ceil(3.2));       // 4.0
println(math.abs(-42));        // 42
```

30. Tensor Reference

Creation Functions

Function	Description	Example
`tensor.zeros(shape)`	Tensor of zeros	`tensor.zeros([3, 3])`
`tensor.ones(shape)`	Tensor of ones	`tensor.ones([2, 4])`
`tensor.from(data)`	Tensor from list	`tensor.from([[1,2],[3,4]])`
`tensor.eye(n)`	Identity matrix	`tensor.eye(4)`
`tensor.rand(shape)`	Random [0,1)	`tensor.rand([3, 3])`
`tensor.arange(start,stop,step)`	Range tensor	`tensor.arange(0, 10, 2)`
`tensor.linspace(start,stop,n)`	Evenly spaced	`tensor.linspace(0, 1, 5)`

Instance Methods

Method	Returns	Description
`.sum()`	float	Sum of all elements
`.mean()`	float	Mean of all elements
`.min()`	float	Minimum element
`.max()`	float	Maximum element
`.argmin()`	int	Index of minimum
`.argmax()`	int	Index of maximum
`.reshape(shape)`	tensor	Reshape to new dimensions
`.transpose()`	tensor	Transpose 2D matrix
`.flatten()`	tensor	Flatten to 1D
`.matmul(other)`	tensor	Matrix multiplication
`.dot(other)`	float	Dot product
`.abs()`	tensor	Element-wise absolute value
`.sqrt()`	tensor	Element-wise square root
`.toList()`	list	Convert to flat list
`.item(idx)`	float	Extract scalar at index
`.slice(start,end)`	tensor	Slice elements
`.shape()`	list	Shape as list of dims
`.ndim`	int	Number of dimensions
`.size`	int	Total number of elements
`.data`	list	Raw data as list

Operators

Tensors support `+`, `-`, `*`, `/` for element-wise arithmetic with broadcasting.

31. AD Reference

Function	Description
`ad.dual(value, deriv)`	Create dual number (value + ε·deriv)
`ad.value(x)`	Extract value component
`ad.deriv(x)`	Extract derivative component
`ad.grad(fn, x)`	Compute gradient of `fn` at `x`
`ad.sin(x)`	Dual-aware sine
`ad.cos(x)`	Dual-aware cosine
`ad.exp(x)`	Dual-aware exponential
`ad.log(x)`	Dual-aware natural log
`ad.sqrt(x)`	Dual-aware square root
`ad.pow(x, n)`	Dual-aware power

32. Pipeline Reference

Entry Point

Function	Description
`pipeline.from(data)`	Create a lazy pipeline from a list or tensor

Transform Operations (chainable)

Method	Description
`.map(fn)`	Apply function to each element
`.filter(fn)`	Keep elements where fn returns true
`.batch(n)`	Group into batches of size n
`.flatten()`	Flatten nested lists
`.take(n)`	Take first n elements
`.skip(n)`	Skip first n elements
`.shuffle()`	Randomly shuffle (Fisher-Yates)
`.zip(other)`	Zip with another list/pipeline

Terminal Operations

Method	Description
`.collect()`	Execute pipeline, return list
`.count()`	Execute pipeline, return count
`.reduce(fn, init)`	Fold elements with function
`.forEach(fn)`	Execute fn on each element

33. Model Reference

Function	Description
`model.save(dict, path)`	Save tensor map to WKTM binary file
`model.load(path)`	Load WKTM file, returns tensor map
`model.meta(dict, key?)`	Get metadata (all or by key)
`model.setMeta(dict, key, val)`	Set metadata key-value pair
`model.loadGGUF(path)`	Import GGUF weights (llama.cpp)
`model.loadONNX(path)`	Import ONNX weights

WKTM Format: `magic("WKTM") + version(u8) + n_tensors(u32) + [name_len(u32) + name + ndim(u8) + shape(i64[]) + data(f64[])]* + meta_len(u32) + json_metadata`

34. FFI Reference

Python (`py`)

Function	Description
`py.available()`	Check if libpython3 is loadable
`py.import(name)`	Import Python module, returns handle
`py.call(mod, method, ...args)`	Call method on module
`py.eval(expr)`	Evaluate Python expression

Java / JVM (`jvm`)

Function	Description
`jvm.available()`	Check if libjvm is loadable (needs JAVA_HOME)
`jvm.import(class)`	Import Java class, returns handle
`jvm.call(cls, method, ...args)`	Call Java method

Node.js (`node`)

Function	Description
`node.available()`	Check if `node` is in PATH
`node.require(module)`	Require npm module, returns handle
`node.eval(expr)`	Evaluate JavaScript expression
`node.call(mod, method, ...args)`	Call method on module

Filesystem (`fs`)

Function	Description
`fs.read(path)`	Read file as string
`fs.write(path, content)`	Write string to file
`fs.exists(path)`	Check if file exists
`fs.args()`	Get CLI arguments as list

35. Complete Examples

Hello World — Basic Language Tour

```
// hello.wk — Demonstrates basic Wakawaka features

// Variables and types
let name = "Wakawaka";
let version = "1.0.0";
let x: int = 42;
let y: float = 3.14;
let flag = true;

println(f"Hello, World!");
println(f"Welcome to {name} v{version}");
println(f"x={x}, y={y}, flag={flag}");

// Collections
let nums = [1, 2, 3, 4, 5];
nums.push(6);
println(f"nums = {nums}");

let sum = nums.reduce(|a, b| a + b, 0);
println(f"sum  = {sum}");

// Functional with closures
let doubled = nums.map(|n| n * 2);
println(f"doubled = {doubled}");

// Maps
let scores = {"Bob": 87, "Alice": 95, "Carol": 92};
for key in scores.keys() {
    println(f"  {key}: {scores.get(key)}");
}

// Control flow
if x > 50 {
    println("x is large");
} else {
    println("x is small");
}

// While loop
var count = 0;
while count < 3 {
    println(f"  count = {count}");
    count += 1;
}

// Range-based for
for i in 1..=5 {
    print(f"{i} ");
}
println("");

// Pattern matching
fn safe_divide(a: float, b: float) -> Result<float, str> {
    if b == 0.0 { return err("division by zero"); }
    return ok(a / b);
}

match safe_divide(84.0, 2.0) {
    ok(v) => println(f"Got value: {v}"),
    err(e) => println(f"Error: {e}"),
}

// Pipe operator
let result = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    |> filter(|n| n % 2 == 0)
    |> map(|n| n ** 2)
    |> sum;
println(f"Sum of squares of even nums 1-10: {result}");

println(f"\n{name} is running!");
```

OOP Demo — Classes, Inheritance, Generics

```
// oop_demo.wk — Object-oriented programming features

// Interface
interface Drawable {
    fn draw(self);
    fn area(self) -> float;
}

// Abstract base class
abstract class Shape {
    var color: str;
    var filled: bool;

    abstract fn area(self) -> float;
    abstract fn perimeter(self) -> float;

    fn describe(self) -> str {
        let fill = self.filled ? "filled" : "outline";
        return f"[{self.color} {fill}] area={self.area()}, perimeter={self.perimeter()}";
    }
}

// Concrete class with inheritance
class Circle extends Shape {
    var radius: float;

    fn new(r: float, color: str) -> Circle {
        return Circle { radius: r, color: color, filled: true };
    }

    override fn area(self) -> float {
        return math.PI * self.radius ** 2;
    }

    override fn perimeter(self) -> float {
        return 2.0 * math.PI * self.radius;
    }

    fn draw(self) {
        println(f"Drawing circle r={self.radius} [{self.color}]");
    }

    // Operator overloading
    fn operator+(self, other: Circle) -> Circle {
        return Circle.new(self.radius + other.radius, self.color);
    }

    fn operator==(self, other: Circle) -> bool {
        return self.radius == other.radius;
    }
}

// Generic class
class Stack<T> {
    var items: []T;

    fn new() -> Stack<T> {
        return Stack { items: [] };
    }

    fn push(var self, item: T) { self.items.push(item); }

    fn pop(var self) -> ?T {
        if self.items.isEmpty() { return none; }
        return some(self.items.pop());
    }

    fn peek(self) -> ?T {
        if self.items.isEmpty() { return none; }
        return some(self.items.last());
    }
```

```
    fn size(self) -> int { return self.items.len(); }
}

// Usage
let c1 = Circle.new(3.0, "red");
let c2 = Circle.new(4.0, "blue");
c1.draw();
println(c1.describe());
println(f"c1 == c2: {c1 == c2}");
println(f"c1 + c2 radius: {(c1 + c2).radius}");

let stack = Stack<int>.new();
stack.push(1);
stack.push(2);
stack.push(3);
println(f"Stack size: {stack.size()}");
println(f"Peek: {stack.peek()}");
println(f"Popped: {stack.pop()}");
```

Concurrency Demo

```
// concurrency.wk — Channels, defer, error handling, closures

// Channels
fn channel_demo() {
    let ch = chan<str>(20);

    fn worker(id: int, ch: chan) {
        for i in 0..3 {
            let msg = f"worker-{id} item-{i}";
            println(f"  [{id}] sending: {msg}");
            ch <- msg;
        }
    }

    go worker(1, ch);
    go worker(2, ch);
    go worker(3, ch);
    sleep(0.5);
    println("Channel demo complete");
}

// Defer
fn defer_demo() {
    fn with_resource(name: str) {
        println(f"  opening {name}");
        defer println(f"  cleanup: {name} closed");
        defer println(f"  cleanup: {name} flushed");
        println(f"  working with {name}");
    }

    with_resource("resource-A");
    with_resource("resource-B");
}

// Error handling with Result
fn error_demo() {
    fn divide(a: float, b: float) -> Result<float, str> {
        if b == 0.0 { return err("division by zero"); }
        return ok(a / b);
    }

    fn parse_and_divide(s: str, divisor: float) -> Result<float, str> {
        let n = float(s);
        if isNone(n) { return err(f"cannot parse '{s}' as float"); }
        return divide(n, divisor);
    }

    let tests = [(10.0, 4.0), (5.0, 0.0), (9.0, 3.0)];
    for t in tests {
```

```
        match divide(t[0], t[1]) {
            ok(v) => println(f"  OK: {t[0]} / {t[1]} = {v}"),
            err(e) => println(f"  Err: {e}"),
        }
    }
}

// Higher-order functions
fn closure_demo() {
    fn make_adder(n: int) -> fn {
        return |x| x + n;
    }

    fn compose(f: fn, g: fn) -> fn {
        return |x| f(g(x));
    }

    let add5 = make_adder(5);
    let mul3 = |x| x * 3;
    println(f"  add5(10) = {add5(10)}");
    println(f"  mul3(10) = {mul3(10)}");
    println(f"  compose(mul3, add5)(10) = {compose(mul3, add5)(10)}");

    // Pipe operator
    let result = 10 |> add5 |> mul3;
    println(f"  10 |> add5 |> mul3 = {result}");
}

println("=== Concurrency Demo ===\n");
println("--- Channels ---");
channel_demo();
println("\n--- Defer ---");
defer_demo();
println("\n--- Error Handling ---");
error_demo();
println("\n--- Closures ---");
closure_demo();
println("\nDemo complete!");
```

Shell Integration Demo

```
// shell_demo.wk — Shell command integration

println("=== Shell Integration ===\n");

// Execute shell commands
let cwd = $`pwd`.trim();
println(f"Current directory: {cwd}");

// Process output as lines
let files = $`ls -la`.lines();
println(f"\nFiles ({files.len()} lines):");
for line in files {
    println(f"  {line}");
}

// Safe variable interpolation
let filename = "hello.dt";
let first_line = $`head -1 examples/{filename}`.trim();
println(f"\nFirst line of {filename}:");
println(first_line);

// Find files
let wk_files = $`find examples -name "*.dt"`.trim().lines();
println(f"\n--- Found {wk_files.len()} .dt files ---");
for f in wk_files {
    println(f"  {f}");
}

// Environment variables
let home = $`echo $HOME`.trim();
println(f"\nHOME = {home}");

println("\nShell integration complete!");
```